Documento demo para Spring AI

Material de prueba para lectura de PDF, división en chunks, generación de embeddings e indexación en un SimpleVectorStore.

Tema	Spring AI y aplicaciones de inteligencia artificial
Idioma	Español
Uso sugerido	Laboratorio de búsqueda semántica y RAG
Contenido	Conceptos, arquitectura, componentes, casos de uso y glosario

> Nota: este documento contiene texto real y variado para que una aplicación pueda extraer contenido, crear fragmentos y recuperar secciones relacionadas mediante similitud semántica.

1. ¿Qué es Spring AI?

Spring AI es un proyecto del ecosistema Spring orientado a simplificar la integración de modelos de inteligencia artificial dentro de aplicaciones Java y Spring Boot. Su propósito es ofrecer abstracciones consistentes para trabajar con modelos de chat, modelos de embeddings, generación de imágenes, herramientas, memoria conversacional y almacenes vectoriales.

La idea central es reducir el acoplamiento entre la aplicación y un proveedor específico. Una aplicación puede usar OpenAI durante una primera etapa y posteriormente incorporar Azure OpenAI, Anthropic, Google Gemini, Mistral u Ollama, manteniendo una arquitectura similar. La configuración, las dependencias y algunas capacidades pueden variar, pero los conceptos principales permanecen.

Spring AI no crea ni entrena un modelo fundacional propio. Actúa como una capa de integración entre la aplicación empresarial y los servicios de inteligencia artificial. Por ello, el desarrollador se concentra en los casos de uso, el diseño de prompts, la seguridad, la recuperación de información y la experiencia del usuario.

Principales objetivos

- Integrar modelos de IA mediante APIs y componentes de Spring.
- Estandarizar el acceso a chat, embeddings, imágenes y herramientas.
- Facilitar patrones como búsqueda semántica, RAG y agentes con herramientas.
- Permitir que la lógica de negocio se mantenga separada del proveedor de IA.

Ejemplo semántico: una consulta con la palabra automóvil debería recuperar textos relacionados con vehículo, carro o coche, aunque no contengan exactamente la misma palabra.

2. Arquitectura conceptual

Una solución con Spring AI suele organizarse en capas. La aplicación recibe una solicitud, prepara el prompt o la consulta, invoca un modelo y procesa la respuesta. Cuando se trabaja con documentos, también intervienen lectores, transformadores de texto, modelos de embeddings y un almacén vectorial.

Componente	Responsabilidad
ChatClient	Construye solicitudes conversacionales y facilita la interacción con un ChatModel.
ChatModel	Representa el modelo generativo que produce respuestas de lenguaje natural.
EmbeddingModel	Convierte textos en vectores numéricos que representan significado.
DocumentReader	Extrae contenido desde PDF, texto, HTML u otras fuentes.
TokenTextSplitter	Divide documentos extensos en fragmentos manejables llamados chunks.
VectorStore	Almacena embeddings y permite recuperar fragmentos similares a una consulta.
Advisor	Agrega comportamientos reutilizables, como memoria o recuperación de context

Flujo básico de documentos: PDF → lectura → documentos → chunks → embeddings → vector store → búsqueda semántica.

3. Lectura de documentos PDF

La lectura de documentos es la primera etapa de una canalización de conocimiento. Un lector de PDF extrae el texto disponible y lo representa mediante objetos Document. Cada objeto puede contener contenido y metadatos, por ejemplo el número de página, el nombre del archivo o una etiqueta de origen.

Los PDF basados en texto son los más apropiados para un laboratorio inicial. Un documento escaneado puede contener solo imágenes y requerir reconocimiento óptico de caracteres. También pueden existir dificultades con documentos protegidos, tablas complejas, columnas múltiples o caracteres mal codificados.

En Spring AI, PagePdfDocumentReader permite leer el contenido por páginas. Esta característica es útil porque los metadatos de página facilitan explicar de dónde provino cada resultado recuperado.

Buenas prácticas para documentos

- Mantener un identificador de origen para cada archivo.
- Conservar metadatos como título, página, categoría y fecha.
- Eliminar contenido repetitivo, encabezados y pies cuando generen ruido.
- Validar que el texto extraído sea legible antes de crear embeddings.
- Evitar indexar datos confidenciales sin controles de seguridad.

Un documento no se convierte directamente en una respuesta. Primero se transforma en unidades de información que luego podrán compararse con la consulta del usuario.

4. División en chunks

Los modelos de embeddings y los modelos generativos tienen límites de tokens. Por esa razón, un documento extenso se divide en fragmentos llamados chunks. Cada chunk debe ser suficientemente pequeño para procesarse con eficiencia, pero también debe conservar contexto suficiente para representar una idea completa.

Un chunk demasiado pequeño puede perder significado. Por ejemplo, una oración aislada podría omitir el sujeto o el tema principal. Un chunk demasiado grande puede mezclar varios asuntos y reducir la precisión de la recuperación. El tamaño adecuado depende del tipo de documento y de las preguntas esperadas.

TokenTextSplitter divide el texto considerando una cantidad aproximada de tokens. En escenarios reales también se puede aplicar superposición entre chunks para conservar continuidad entre un fragmento y el siguiente.

Estrategia	Ventaja	Riesgo
Chunks pequeños	Mayor precisión temática	Pérdida de contexto
Chunks medianos	Equilibrio entre contexto y precisión	Requiere pruebas
Chunks grandes	Conservan más información	Pueden mezclar temas
Superposición	Mantiene continuidad	Incrementa almacenamiento y costo

Para validar la estrategia, conviene formular preguntas concretas y revisar si los fragmentos recuperados contienen la información necesaria sin demasiado texto irrelevante.

5. Embeddings y SimpleVectorStore

Un embedding es una representación numérica de un texto. Textos con significados cercanos suelen producir vectores cercanos dentro de un espacio matemático. Esta propiedad permite comparar una consulta con los chunks almacenados.

El modelo text-embedding-3-small de OpenAI puede utilizarse para generar embeddings de cada fragmento. Durante la indexación, la aplicación envía el contenido al modelo de embeddings, recibe un vector y lo almacena junto con el texto y sus metadatos.

SimpleVectorStore es un almacén vectorial apropiado para demostraciones y pruebas. Mantiene la información en memoria y permite realizar búsquedas por similitud. No es la opción recomendada para producción cuando se necesita alta disponibilidad, persistencia, grandes volúmenes o búsqueda distribuida.

Proceso de indexación

1. Leer el documento PDF.
2. Extraer contenido y metadatos.
3. Dividir el texto en chunks.
4. Generar un embedding por cada chunk.
5. Guardar texto, vector y metadatos en el VectorStore.

Importante: llamar a vectorStore.add(documentos) implica generar embeddings. Por tanto, la indexación consume la API del proveedor de embeddings.

6. Búsqueda semántica

La búsqueda semántica compara el significado de la consulta con el significado de los fragmentos indexados. A diferencia de una búsqueda por palabras exactas, puede recuperar conceptos relacionados, sinónimos y expresiones equivalentes.

Cuando el usuario escribe una pregunta, la aplicación genera un embedding de esa consulta. Luego compara dicho vector con los vectores del almacén. Los resultados se ordenan según su similitud y se devuelven los fragmentos más cercanos.

Dos parámetros comunes son topK y similarityThreshold. topK limita la cantidad máxima de resultados. El umbral de similitud descarta resultados poco relacionados. Un umbral demasiado alto puede ocultar información útil; uno muy bajo puede introducir ruido.

Consulta	Resultado esperado
¿Qué es Spring AI?	Fragmentos que describen el propósito y las abstracciones del proyecto.
¿Cómo se divide un PDF?	Sección sobre chunks y TokenTextSplitter.
¿Dónde se guardan los vectores?	Sección sobre SimpleVectorStore y almacenamiento.
¿Qué significa topK?	Sección sobre recuperación de los resultados más similares.

7. Recuperación aumentada por generación (RAG)

RAG combina recuperación de información y generación de lenguaje. Primero busca fragmentos relevantes en un almacén vectorial. Después incorpora esos fragmentos como contexto de un prompt enviado a un modelo generativo.

El propósito es producir respuestas apoyadas en documentos específicos, en lugar de depender únicamente del conocimiento general del modelo. Una implementación responsable debe mostrar las fuentes, limitar el contexto y evitar afirmar información que no esté respaldada por los documentos recuperados.

En una arquitectura RAG, la indexación suele ejecutarse antes de las consultas. La recuperación y la generación se ejecutan durante cada pregunta. Es importante distinguir ambas fases para controlar costos y diagnosticar errores.

Fase	Actividad
Preparación	Lectura, limpieza, chunking, embeddings e indexación.
Consulta	Embedding de la pregunta y búsqueda semántica.
Contexto	Selección de los chunks más relevantes.
Generación	El ChatModel responde utilizando el contexto recuperado.
Validación	Revisión de precisión, cobertura y trazabilidad.

Una búsqueda semántica devuelve fragmentos. Un sistema RAG utiliza esos fragmentos para construir una respuesta final.

8. Casos de uso y glosario

Casos de uso

- Asistente para consultar manuales técnicos.
- Buscador inteligente de políticas y procedimientos.
- Atención de preguntas sobre cursos, sílabos y materiales académicos.
- Consulta de contratos, informes y documentos institucionales.
- Recuperación de conocimiento desde una base documental interna.

Glosario

Término	Definición
Chunk	Fragmento de texto obtenido al dividir un documento.
Embedding	Vector numérico que representa características semánticas de un texto.
VectorStore	Componente que almacena vectores y permite búsquedas por similitud.
Top K	Cantidad máxima de resultados recuperados.
Metadatos	Información adicional del documento, como página, archivo o categoría.
RAG	Patrón que combina recuperación de información y generación de respuestas.
Prompt	Instrucción o conjunto de mensajes enviados a un modelo generativo.
Modelo de chat	Modelo que genera una respuesta de lenguaje natural.

Preguntas sugeridas para probar la búsqueda semántica: ¿qué componente convierte textos en vectores?, ¿por qué se divide un documento en chunks?, ¿cuál es la diferencia entre búsqueda semántica y RAG?, ¿qué limitaciones tiene SimpleVectorStore?